L A B R E P O R T · 0 2

University of Azad Jammu and Kashmir

Department of Software Engineering

Digital Image Processing Lab

Lab Task #02

Digital Images, Pixels, Color, and Basic Transformations

Virtual environment · Pixels and channels · Crop and resize · Histogram · Bit depth

STUDENT NAME	Sheikh Haseeb-ur-Rehman
ROLL NUMBER	2024-SE-31
INSTRUCTOR	Engr. Fahad Nisar
SEMESTER	Fifth
SESSION	2024-28 (SE)
DATE	October 29, 2026

1 Project and Virtual Environment

Objective: Create the dip-lab-02 project, build an isolated virtual environment, install the required libraries, and verify that the environment and the notebook work correctly.

Create the Project and Virtual Environment

▸ **Result**

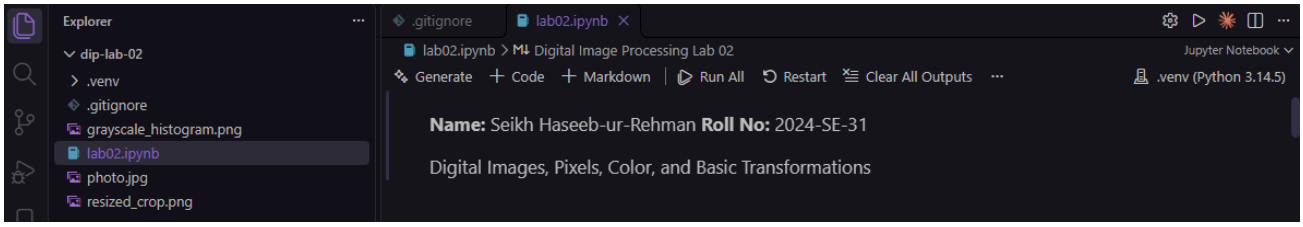


Figure 1 — Project folder and .venv in VS Code

▸ **Code**

```
mkdir dip-lab-02
cd dip-lab-02
python -m venv .venv
```

Listing 1 — Creating the project folder and virtual environment

Activate the Environment

▸ **Result**

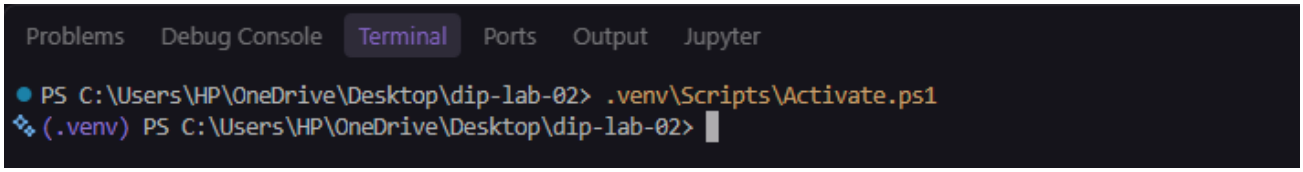


Figure 2 — Virtual environment activated

▸ **Code**

```
# PowerShell
.venv\Scripts\Activate.ps1

# Command Prompt (alternative)
.venv\Scripts\activate.bat
```

Listing 2 — Activation commands

Install the Required Packages

▸ **Result**

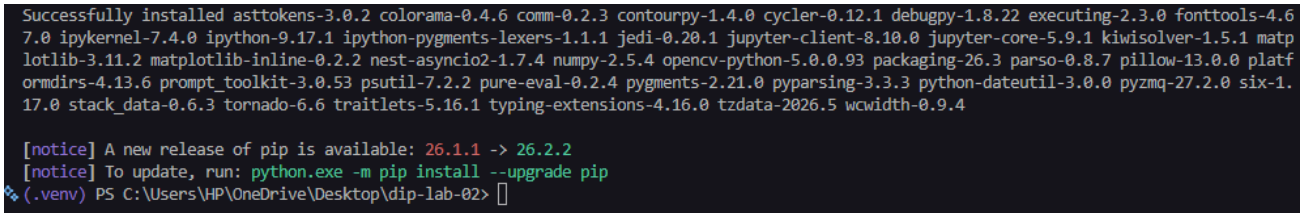


Figure 3 — Package installation inside the environment

▸ **Code**

```
pip install numpy opencv-python matplotlib ipykernel
```

Listing 3 — Installing the libraries

ipykernel is installed in addition to the three required libraries because it is needed to run the notebook (.ipynb) in VS Code.

■ Verify the Imports and the Python Executable

▸ **Result**

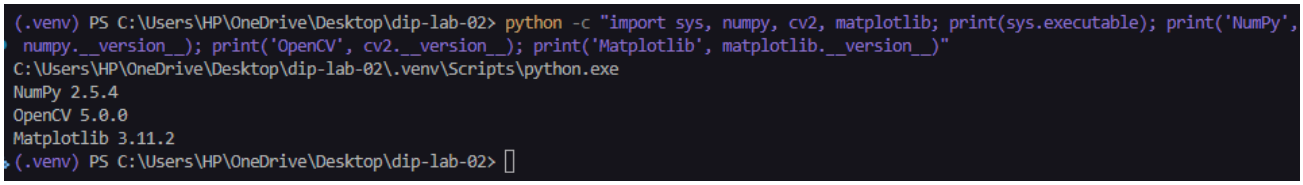


Figure 4 — Import and executable verification

▸ **Code**

```
python -c "import sys, numpy, cv2, matplotlib; print(sys.executable); print('NumPy', numpy.__version__);
print('OpenCV', cv2.__version__); print('Matplotlib', matplotlib.__version__)"
```

Listing 4 — Verifying imports and the interpreter path

■ Add .venv/ to .gitignore

▸ **Result**



Figure 5 — Contents of .gitignore

▸ **Code**

```
.venv/
```

Listing 5 — .gitignore entry

■ Notebook Environment Verification

▸ **Result**

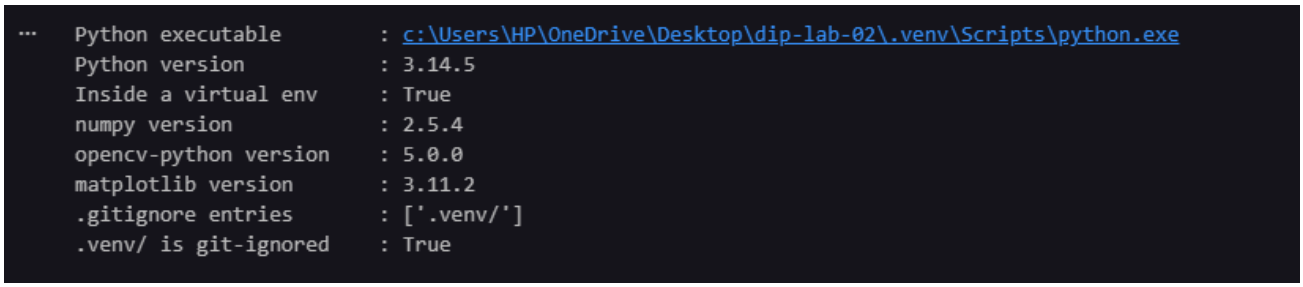


Figure 6 — Environment check inside the notebook

▸ **Code**

```
import sys
import platform


print("Python executable        :", sys.executable)
print("Python version           :", platform.python_version())
print("Inside a virtual env     :", sys.prefix != sys.base_prefix)


import numpy
import cv2
import matplotlib
print("numpy version            :", numpy.__version__)
print("opencv-python version    :", cv2.__version__)
print("matplotlib version       :", matplotlib.__version__)


with open(".gitignore") as f:
    entries = [line.strip() for line in f if line.strip()]
print(".gitignore entries       :", entries)
print(".venv/ is git-ignored    :", ".venv/" in entries)
```

Listing 6 — Notebook cell: environment verification

2 Load and Inspect the Image

Objective: Load the input image with OpenCV, validate that it was read correctly, and inspect its dimensions, pixel values, and value ranges.

■ Load the Image

▸ **Result**

```
Image shape     : (1792, 2400, 3)
Image data type : uint8
```

Figure 7 — Image shape and data type

▸ **Code**

```
import os
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt


IMAGE_NAME = "photo.jpg"


image = cv.imread(IMAGE_NAME)  # OpenCV loads color images in BGR order
if image is None:
    raise FileNotFoundError(f"Image not found or unreadable: {IMAGE_NAME}")



print("Image shape     :", image.shape)
print("Image data type :", image.dtype)
```

Listing 7 — Loading the image with cv.imread()

■ Inspect Dimensions, Pixels, and Value Ranges

▸ **Result**

```
Height (rows)      : 1792
Width (columns)    : 2400
Number of channels : 3
Top-left pixel [0, 0] (B, G, R)     : [35 23 43]
Center pixel [896, 1200] (B, G, R) : [  4 139 183]
Global minimum value : 0
Global maximum value : 255
Blue  channel min / max : 0 / 255
Green channel min / max : 0 / 255
Red   channel min / max : 0 / 255
```

Figure 8 — Image properties and pixel values

▸ **Code**

```
height, width, channels = image.shape
print("Height (rows)      :", height)
print("Width (columns)    :", width)
print("Number of channels :", channels)

if height < 300 or width < 300:
    raise ValueError("Image must be at least 300 x 300 pixels.")

cy, cx = height // 2, width // 2
# Pixels are indexed [row, column] = [y, x]; OpenCV order is (B, G, R)
print("Top-left pixel [0, 0] (B, G, R)     :", image[0, 0])
print(f"Center pixel [{cy}, {cx}] (B, G, R) :", image[cy, cx])

print("Global minimum value :", image.min())
print("Global maximum value :", image.max())
print("Blue  channel min / max :", image[:, :, 0].min(), "/", image[:, :, 0].max())
print("Green channel min / max :", image[:, :, 1].min(), "/", image[:, :, 1].max())
print("Red   channel min / max :", image[:, :, 2].min(), "/", image[:, :, 2].max())
```

Listing 8 — Inspecting the image

Pixels are indexed as [row, column] = [y, x], and OpenCV stores each pixel in (B, G, R) order. The values measured for photo.jpg are summarised below.

Property	Value
Height × Width × Channels	1792 × 2400 × 3
Data type	uint8
Top-left pixel [0, 0] (B, G, R)	[35 23 43]
Center pixel [896, 1200] (B, G, R)	[4 139 183]
Global minimum / maximum	0 / 255
Blue, Green, Red minimum / maximum	0 / 255 for each channel

3 Color Conversion and Channels

Objective: Convert the image from BGR to RGB, create a grayscale version, separate the three color channels, and convert the center pixel to a HEX value.

■ Six-View Figure and Center Pixel HEX Value

▸ **Result**

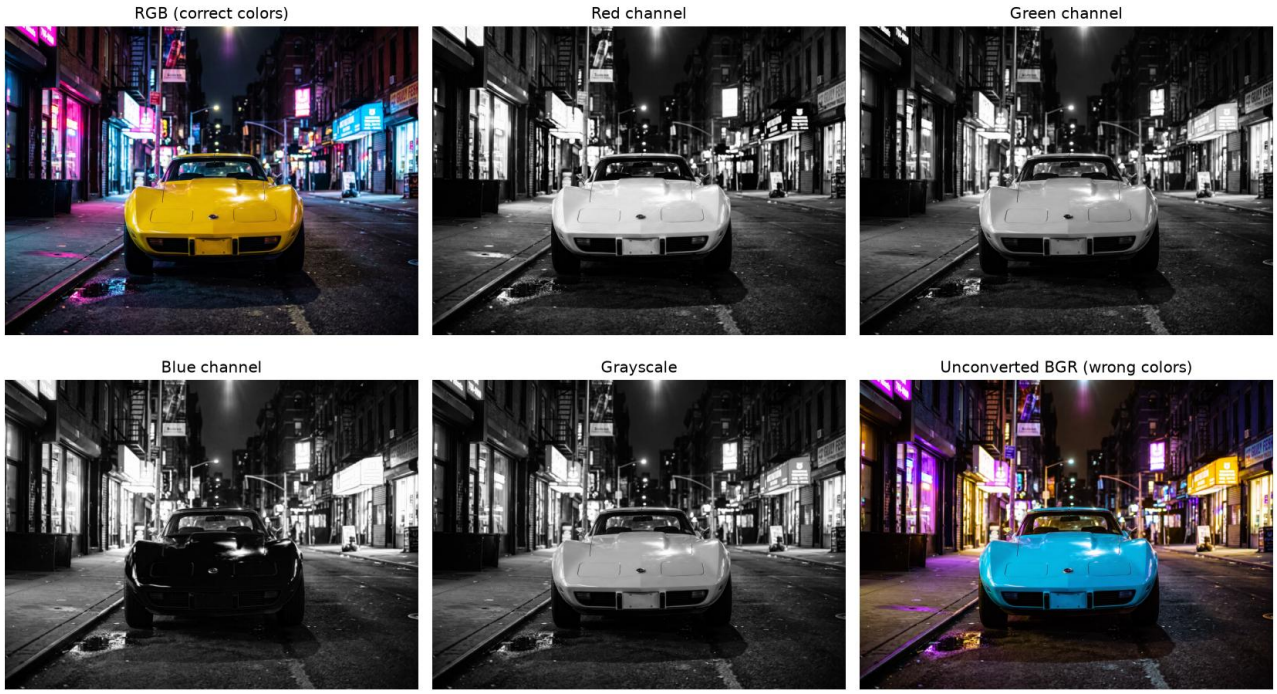


Figure 9 — RGB, channels, grayscale, and unconverted BGR display

```
RGB shape       : (1792, 2400, 3)
Grayscale shape : (1792, 2400)

Center pixel RGB : (183, 139, 4)
Center pixel HEX : #B78B04
```

Figure 10 — Shapes and center pixel HEX output

Center pixel	Value
BGR (OpenCV order)	(4, 139, 183)
RGB	(183, 139, 4)
HEX	#B78B04

▸ **Code**

```
rgb = cv.cvtColor(image, cv.COLOR_BGR2RGB)
gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
r, g, b = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]


print("RGB shape       :", rgb.shape)
print("Grayscale shape :", gray.shape)


fig, axes = plt.subplots(2, 3, figsize=(14, 8))
panels = [
    (rgb, "RGB (correct colors)", None),
    (r, "Red channel", "gray"),
    (g, "Green channel", "gray"),
    (b, "Blue channel", "gray"),
    (gray, "Grayscale", "gray"),
    (image, "Unconverted BGR (wrong colors)", None),
]
for ax, (img, title, cmap) in zip(axes.ravel(), panels):
    ax.imshow(img, cmap=cmap)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.show()


# Center pixel RGB -> HEX
cr, cg, cb = (int(v) for v in rgb[cy, cx])
hex_value = f"#{cr:02X}{cg:02X}{cb:02X}"
print(f"Center pixel RGB : ({cr}, {cg}, {cb})")
print("Center pixel HEX :", hex_value)
```

Listing 9 — Color conversion, channel separation, and HEX conversion

■ Explanation

BGR versus RGB: OpenCV stores the three color channels of each pixel in the order blue, green, red, while Matplotlib expects red, green, blue. If a BGR image is shown directly with Matplotlib, red and blue are swapped, so the yellow car looks blue-cyan in the last panel of the figure. cv.cvtColor() with COLOR_BGR2RGB reorders the channels so the colors display correctly, and the reverse conversion is used before saving with cv.imwrite().

Channels: Each channel is a grayscale image showing how much of that color is present at every pixel (brighter means more). The car is bright in the red and green channels and dark in the blue channel, because yellow is made of red and green light with almost no blue. The center pixel agrees with this: blue is only 4, while green is 139 and red is 183.

4 Crop, Resize, and Save

Objective: Crop an exact 100 × 100 region, resize it to 200 × 200 pixels, display the three images side by side, and save the result as resized_crop.png.

■ Crop, Resize, and Display

▸ **Result**

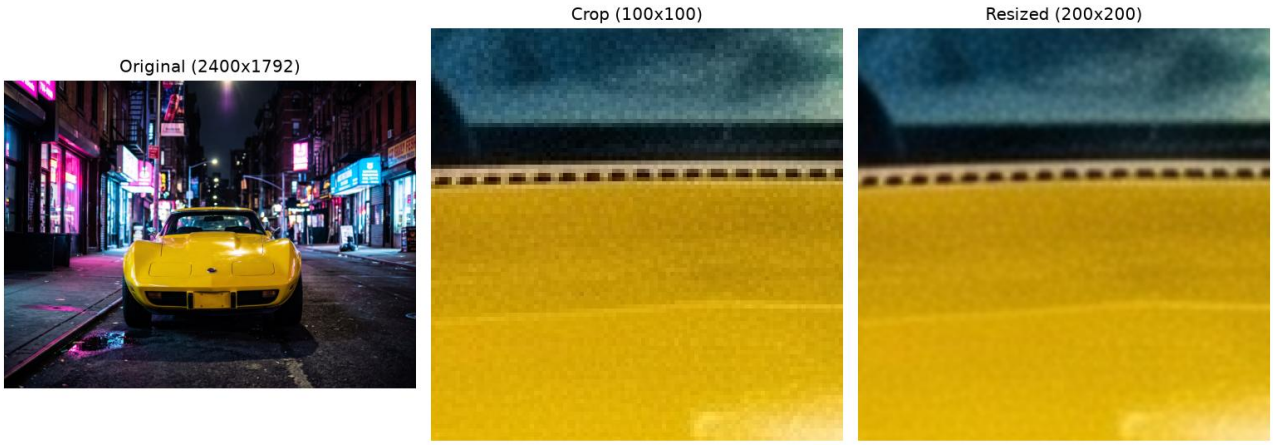


Figure 11 — Original, crop, and resized image

```
Crop region : rows 846:946, cols 1150:1250
Crop shape  : (100, 100, 3)
Resized shape: (200, 200, 3)

cv.imwrite() saved resized_crop.png successfully: True
```

Figure 12 — Crop, resize, and save output

Item	Value
Crop region	rows 846:946, columns 1150:1250
Crop shape	(100, 100, 3)
Resized shape	(200, 200, 3)
Saved file / cv.imwrite() result	resized_crop.png / True

▸ **Code**

```
# Exact 100 x 100 crop centered on the image: image[y_start:y_end, x_start:x_end]
y_start, y_end = cy - 50, cy + 50
x_start, x_end = cx - 50, cx + 50
crop = rgb[y_start:y_end, x_start:x_end]   # crop the RGB copy so Matplotlib shows correct colors
print(f"Crop region : rows {y_start}:{y_end}, cols {x_start}:{x_end}")
print("Crop shape  :", crop.shape)
assert crop.shape[:2] == (100, 100), "Crop is not 100 x 100"


# cv.resize takes (width, height)
resized = cv.resize(crop, (200, 200), interpolation=cv.INTER_LINEAR)
print("Resized shape:", resized.shape)
assert resized.shape[:2] == (200, 200), "Resize is not 200 x 200"


fig, axes = plt.subplots(1, 3, figsize=(13, 5))
for ax, (img, title) in zip(
    axes,
    [(rgb, f"Original ({width}x{height})"), (crop, "Crop (100x100)"), (resized, "Resized (200x200)")],
):
    ax.imshow(img)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.show()


# Convert RGB back to BGR before saving with OpenCV
resized_bgr = cv.cvtColor(resized, cv.COLOR_RGB2BGR)
saved = cv.imwrite("resized_crop.png", resized_bgr)
print("cv.imwrite() saved resized_crop.png successfully:", saved)
```

Listing 10 — Cropping, resizing, displaying, and saving

The crop is taken from the RGB copy so that Matplotlib shows the correct colors. The resized image is converted back to BGR before cv.imwrite(), because OpenCV expects BGR when writing files.

5 Grayscale Histogram

Objective: Flatten the grayscale image, plot a 256-bin histogram over the range 0–256, save it as grayscale_histogram.png, and interpret it.

■ Histogram

▸ **Result**

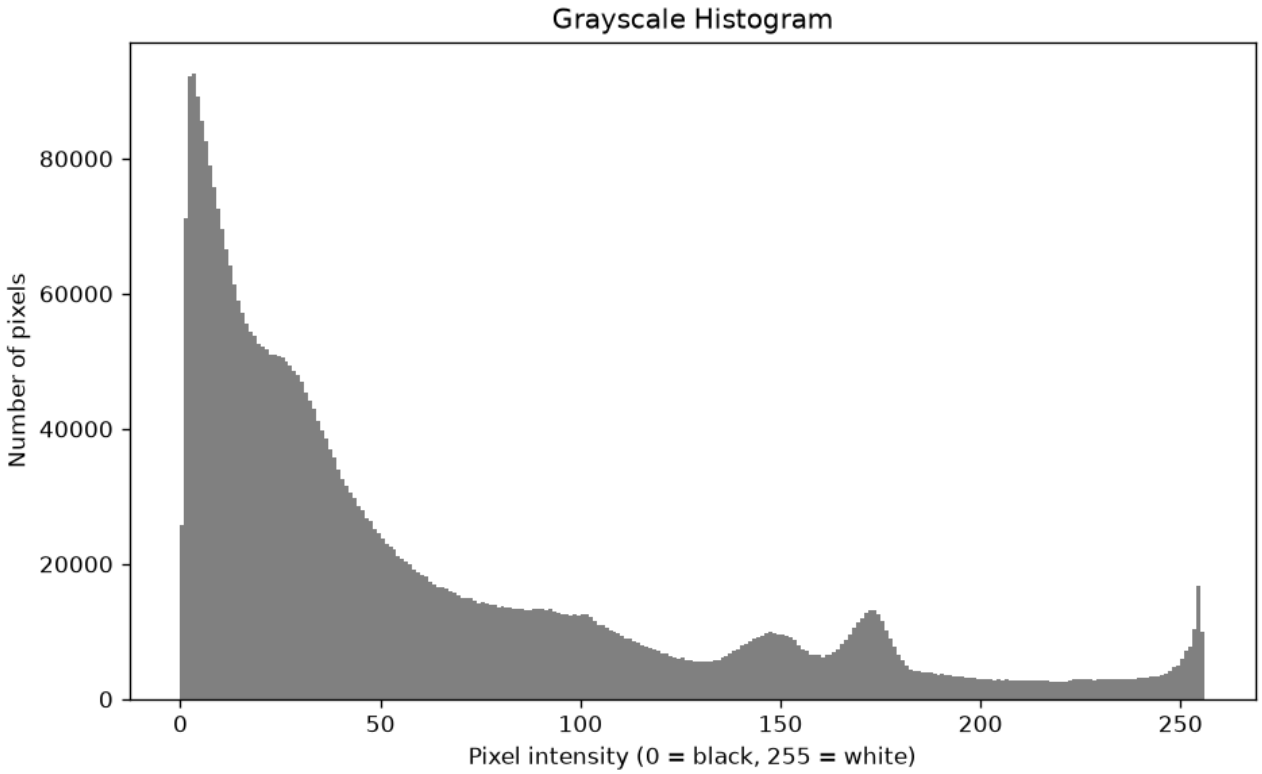


Figure 13 — Grayscale histogram of the image

```
Mean intensity          : 61.7
Dark pixels   (0-84)    : 73.1%
Mid pixels    (85-169)  : 17.7%
Bright pixels (170-255) : 9.2%

Flattened grayscale size: 4300800
```

Figure 14 — Histogram statistics

Measure	Value
Mean intensity	61.7 (out of 255)
Dark pixels (0–84)	73.1%
Mid-tone pixels (85–169)	17.7%
Bright pixels (170–255)	9.2%

▸ **Code**

```
flat = gray.flatten()
print("Flattened grayscale size:", flat.size)


fig = plt.figure(figsize=(8, 5))
plt.hist(flat, bins=256, range=(0, 256), color="gray")
plt.title("Grayscale Histogram")
plt.xlabel("Pixel intensity (0 = black, 255 = white)")
plt.ylabel("Number of pixels")
plt.tight_layout()
fig.savefig("grayscale_histogram.png", dpi=120)
plt.show()


# Numbers to support your written interpretation
print(f"Mean intensity          : {flat.mean():.1f}")
print(f"Dark pixels   (0-84)    : {np.mean(flat < 85) * 100:.1f}%")
print(f"Mid pixels    (85-169)  : {np.mean((flat >= 85) & (flat < 170)) * 100:.1f}%")
print(f"Bright pixels (170-255) : {np.mean(flat >= 170) * 100:.1f}%")
```

Listing 11 — Creating and saving the grayscale histogram

■ Interpretation

Is the image dark, bright, or balanced? The image is mainly dark. Its mean intensity is 61.7 out of 255, and 73.1% of the pixels fall in the dark range, compared with 17.7% mid-tones and only 9.2% bright pixels. The large peak near 0–10 comes from the night sky, shadows, and road, while the small peaks near 150–175 and the spike at 255 come from the yellow car and the bright neon lights.

What a histogram shows and does not show: A histogram counts how many pixels have each intensity value, so it reveals overall brightness, contrast, and whether highlights or shadows are clipped. It does not show where the pixels are located, what objects are in the image, or any color information, so very different images can have the same histogram.

6 Bit Depth

Objective: Create an 8-bit grayscale gradient, quantize it to 1-, 2-, and 4-bit intensity levels, compare the results, and confirm the number of unique values.

■ Quantized Gradients

▸ **Result**

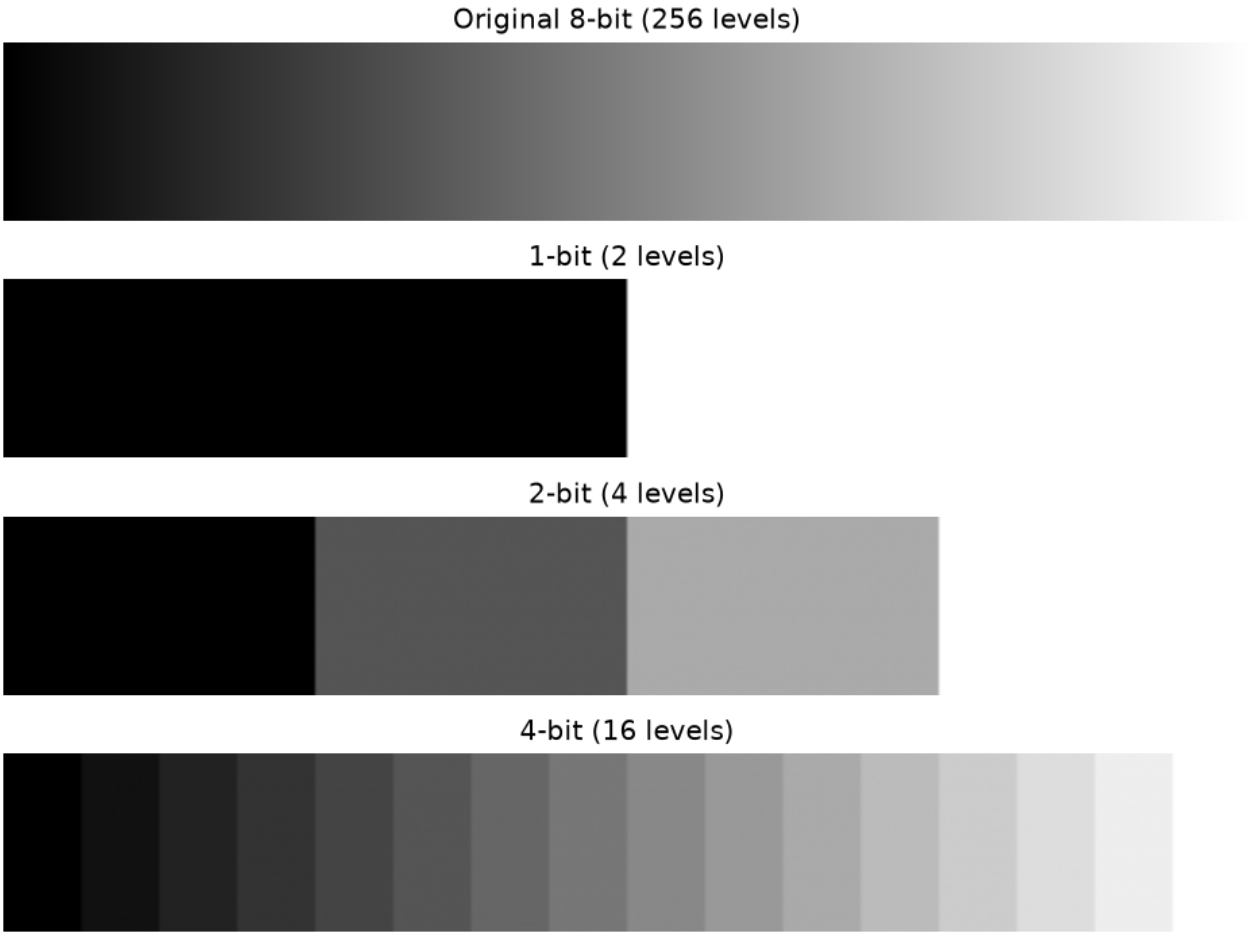


Figure 15 — Bit-depth comparison

```
Unique values in 8-bit gradient : 256
Unique values in 1-bit gradient : 2
Unique values in 2-bit gradient : 4
Unique values in 4-bit gradient : 16
```

Figure 16 — Unique-value counts

Bit depth	Intensity levels	Unique values found
8-bit (original)	256	256
1-bit	2	2
2-bit	4	4
4-bit	16	16

▸ **Code**

```
# 8-bit horizontal gradient 0..255 (256 wide, 80 tall)
gradient = np.tile(np.arange(256, dtype=np.uint8), (80, 1))


def quantize(img, bits):
    """Reduce to 2**bits intensity levels, spread over 0..255."""
    levels = 2 ** bits
    step = 256 // levels
    scale = 255 // (levels - 1)
    return ((img // step) * scale).astype(np.uint8)


q1, q2, q4 = quantize(gradient, 1), quantize(gradient, 2), quantize(gradient, 4)
versions = [
    (gradient, "Original 8-bit (256 levels)"),
    (q1, "1-bit (2 levels)"),
    (q2, "2-bit (4 levels)"),
    (q4, "4-bit (16 levels)"),
]

fig, axes = plt.subplots(4, 1, figsize=(8, 6))
for ax, (img, title) in zip(axes, versions):
    ax.imshow(img, cmap="gray", vmin=0, vmax=255, aspect="auto")
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.show()


print("Unique values in 8-bit gradient :", len(np.unique(gradient)))
print("Unique values in 1-bit gradient :", len(np.unique(q1)))
print("Unique values in 2-bit gradient :", len(np.unique(q2)))
print("Unique values in 4-bit gradient :", len(np.unique(q4)))
```

Listing 12 — Quantizing the gradient and counting unique values

■ Resolution versus Bit Depth

Resolution is the number of pixels in an image (width × height, here 2400 × 1792), so it controls how much spatial detail the image can hold. Bit depth is the number of bits used to store each pixel's intensity, so it controls how many different brightness levels a pixel can have: 1 bit gives 2 levels, 2 bits give 4, 4 bits give 16, and 8 bits give 256. Lowering the resolution makes the image smaller or blocky, while lowering the bit depth makes smooth gradients break into visible bands, as the figure shows.

7 Final Run and Project Completion

Objective: Run the complete project from the activated environment without errors, confirm that all required files exist, and deactivate the environment.

■ Final Check

▸ **Result**

```
photo.jpg                 exists: True
resized_crop.png          exists: True
grayscale_histogram.png   exists: True
.gitignore                exists: True
resized_crop.png shape    : (200, 200, 3)
resized_crop.png is 200x200: True
All required files present : True
```

Figure 17 — Final successful run

▸ **Code**

```
import os
import cv2 as cv


required = ["photo.jpg", "resized_crop.png", "grayscale_histogram.png", ".gitignore"]
for name in required:
    print(f"{name:<26} exists:", os.path.isfile(name))


check = cv.imread("resized_crop.png")
print("resized_crop.png shape     :", None if check is None else check.shape)
print("resized_crop.png is 200x200:", check is not None and check.shape[:2] == (200, 200))
print("All required files present :", all(os.path.isfile(n) for n in required))
```

Listing 13 — Final check of the required output files

■ Deactivate the Environment

▸ **Result**

```
PS C:\Users\HP\OneDrive\Desktop\dip-lab-02> .venv\Scripts\Activate.ps1
(.venv) PS C:\Users\HP\OneDrive\Desktop\dip-lab-02> deactivate
PS C:\Users\HP\OneDrive\Desktop\dip-lab-02>
```

Figure 18 — Environment deactivated

▸ **Code**

```
deactivate
```

Listing 14 — Leaving the virtual environment

■ Submitted Files

File	Description
lab02.ipynb	Lab implementation (Steps 1–8)
photo.jpg	Input image (2400 × 1792, color)
resized_crop.png	Saved 200 × 200 resized crop
grayscale_histogram.png	Saved grayscale histogram
.gitignore	Contains .venv/
2024-SE-31-LAB02.docx	This report

The .venv folder is not submitted.